

Министерство науки и высшего образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЁТ

по лабораторной работе №5
по курсу «Логика и основы алгоритмизации в инженерных задачах»
на тему «Определение характеристик графов»

Выполнили:
студенты группы 24ВВВЗ
Задоркин Тимофей
Комиссаров Андрей

Приняли:
Юрова О.В.
Деев М.В.

Пенза 2025

Цель – познакомиться с характеристиками графа и научиться их определять с помощью программы. Познакомиться с алгоритмом задания графа через матрицы.

Общие сведения.

Если G – граф (рисунок 1), содержащий непустое множество n вершин V и множество ребер E , где $e(v_i, v_j)$ – ребро между двумя произвольными вершинами v_i и v_j , тогда размер графа G есть мощность множества ребер $|E(G)|$ или, количество ребер графа.

Степенью вершины графа G называется число инцидентных ей ребер. Степень вершины v_i обозначается через $\deg(v_i)$.

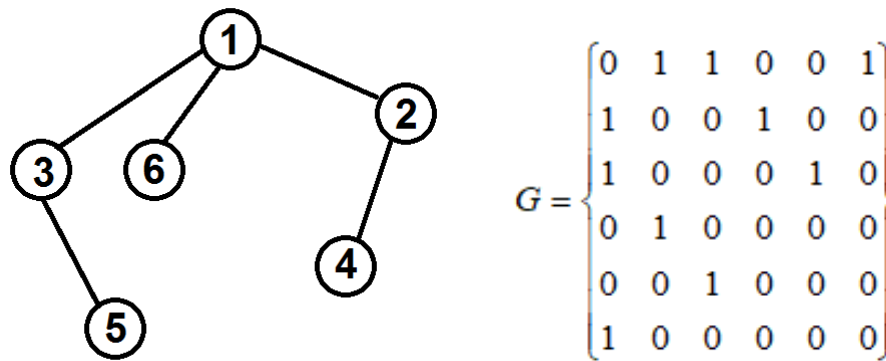


Рисунок – Граф

Вершина v_i со степенью 0 называется изолированной, со степенью 1 – концевой.

Вершина графа, смежная с каждой другой его вершиной, называется доминирующей

Задание 1

1. Сгенерируйте (используя генератор случайных чисел) матрицу смежности для неориентированного графа G . Выведите матрицу на экран.
2. Определите размер графа G , используя матрицу смежности графа.
3. Найдите изолированные, концевые и доминирующие вершины.

Задание 2*

1. Постройте для графа G матрицу инцидентности.
2. Определите размер графа G , используя матрицу инцидентности графа.
3. Найдите изолированные, концевые и доминирующие вершины.

Код программы

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int** create_adjacency_matrix(int n, double edgeProbability) {
    int** matrix = (int**)malloc(n * sizeof(int*));
    for (int i = 0; i < n; i++) {
        matrix[i] = (int*)malloc(n * sizeof(int));
    }
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (i == j) {
                matrix[i][j] = 0;
            }
            else if (i < j) {
                double randomValue = (double)rand() / RAND_MAX;
                if (randomValue < edgeProbability) {
                    matrix[i][j] = 1;
                    matrix[j][i] = 1;
                }
            }
            else {
                matrix[i][j] = 0;
                matrix[j][i] = 0;
            }
        }
    }
}
```

```

        return matrix;
    }

int** create_incidence_matrix(int** adj_matrix, int n, int*
edge_count) {
    *edge_count = 0;
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (adj_matrix[i][j] == 1) {
                (*edge_count)++;
            }
        }
    }

    int** inc_matrix = (int**)malloc(n * sizeof(int*));
    for (int i = 0; i < n; i++) {
        inc_matrix[i] = (int*)malloc(*edge_count * sizeof(int));
    }

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < *edge_count; j++) {
            inc_matrix[i][j] = 0;
        }
    }

    int edge_index = 0;
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (adj_matrix[i][j] == 1) {
                inc_matrix[i][edge_index] = 1;
                inc_matrix[j][edge_index] = 1;
                edge_index++;
            }
        }
    }

    return inc_matrix;
}

```

```

void print_incidence_matrix(int** inc_matrix, int n, int
edge_count) {
    printf("\nМатрица инцидентности (%d вершин x %d ребер):\n",
n, edge_count);
    printf("    ");
    for (int j = 0; j < edge_count; j++) {
        printf("e%-2d", j + 1);
    }
    printf("\n");
    for (int i = 0; i < n; i++) {
        printf("v%-2d", i + 1);
        for (int j = 0; j < edge_count; j++) {
            printf("%2d ", inc_matrix[i][j]);
        }
        printf("\n");
    }
}

int vertex_degree_from_incidence(int** inc_matrix, int n, int
edge_count, int vertex) {
    int degree = 0;
    for (int j = 0; j < edge_count; j++) {
        if (inc_matrix[vertex][j] == 1) {
            degree++;
        }
    }
    return degree;
}

void find_isolated_vertices_from_incidence(int** inc_matrix, int
n, int edge_count) {
    printf("Изолированные вершины: ");
    int has_isolated = 0;
    for (int i = 0; i < n; i++) {
        int is_isolated = 1;
        for (int j = 0; j < edge_count; j++) {

```

```

        if (inc_matrix[i][j] == 1) {
            is_isolated = 0;
            break;
        }
    }
    if (is_isolated) {
        printf("%d ", i + 1);
        has_isolated = 1;
    }
}

if (!has_isolated) printf("нет");
printf("\n");
}

void find_end_vertices_from_incidence(int** inc_matrix, int n,
int edge_count) {
    printf("Концевые вершины: ");
    int has_end = 0;
    for (int i = 0; i < n; i++) {
        int edge_count_for_vertex = 0;
        for (int j = 0; j < edge_count; j++) {
            if (inc_matrix[i][j] == 1) {
                edge_count_for_vertex++;
            }
        }
        if (edge_count_for_vertex == 1) {
            printf("%d ", i + 1);
            has_end = 1;
        }
    }
    if (!has_end) printf("нет");
    printf("\n");
}

```

```

void find_dominant_vertices_from_incidence(int** inc_matrix, int
n, int edge_count) {
    printf("Доминирующие вершины: ");
    int has_dominant = 0;
    for (int i = 0; i < n; i++) {
        int edge_count_for_vertex = 0;
        for (int j = 0; j < edge_count; j++) {
            if (inc_matrix[i][j] == 1) {
                edge_count_for_vertex++;
            }
        }
        if (edge_count_for_vertex == n - 1) {
            printf("%d ", i + 1);
            has_dominant = 1;
        }
    }
    if (!has_dominant) printf("нет");
    printf("\n");
}

int graph_size(int** matrix, int n) {
    int edges = 0;
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (matrix[i][j] == 1) {
                edges++;
            }
        }
    }
    return edges;
}

void print_matrix(int** matrix, int n) {
    printf("Матрица смежности (%dx%d):\n", n, n);
    printf("    ");

```

```

    for (int i = 0; i < n; i++) {
        printf("%2d ", i + 1);
    }
    printf("\n");
    for (int i = 0; i < n; i++) {
        printf("%2d ", i + 1);
        for (int j = 0; j < n; j++) {
            printf("%2d ", matrix[i][j]);
        }
        printf("\n");
    }
}

int vertex_degree(int** matrix, int n, int vertex) {
    int degree = 0;
    for (int i = 0; i < n; i++) {
        if (matrix[vertex][i] == 1) {
            degree++;
        }
    }
    return degree;
}

void print_vertex_degrees_from_incidence(int** inc_matrix, int
n, int edge_count) {
    printf("\nСтепени вершин:\n");
    for (int i = 0; i < n; i++) {
        int degree = vertex_degree_from_incidence(inc_matrix, n,
edge_count, i);
        printf("Вершина %d: степень %d\n", i + 1, degree);
    }
}

void find_special_vertices(int** matrix, int n) {
    printf("\nСпециальные вершины (через матрицу
смежности):\n");

```



```

printf("Изолированные вершины (степень 0): ");
int has_isolated = 0;
for (int i = 0; i < n; i++) {
    if (vertex_degree(matrix, n, i) == 0) {
        printf("%d ", i + 1);
        has_isolated = 1;
    }
}
if (!has_isolated) printf("нет");
printf("\n");
printf("Концевые вершины (степень 1): ");
int has_end = 0;
for (int i = 0; i < n; i++) {
    if (vertex_degree(matrix, n, i) == 1) {
        printf("%d ", i + 1);
        has_end = 1;
    }
}
if (!has_end) printf("нет");
printf("\n");
printf("Доминирующие вершины (степень %d): ", n - 1);
int has_dominant = 0;
for (int i = 0; i < n; i++) {
    if (vertex_degree(matrix, n, i) == n - 1) {
        printf("%d ", i + 1);
        has_dominant = 1;
    }
}
if (!has_dominant) printf("нет");
printf("\n");
}

void print_vertex_degrees(int** matrix, int n) {

```

```

printf("\nСтепени вершин (через матрицу смежности):\n");
for (int i = 0; i < n; i++) {
    int degree = vertex_degree(matrix, n, i);
    printf("Вершина %d: степень %d\n", i + 1, degree);
}
}

void free_matrix(int** matrix, int n) {
    for (int i = 0; i < n; i++) {
        free(matrix[i]);
    }
    free(matrix);
}

void free_incidence_matrix(int** matrix, int n) {
    for (int i = 0; i < n; i++) {
        free(matrix[i]);
    }
    free(matrix);
}

void show_menu() {
    printf("\n=== МЕНЮ ===\n");
    printf("1 - Показать матрицу смежности\n");
    printf("2 - Вывести размер графа G, изолированные, концевые  
и доминирующие вершины (через матрицу смежности)\n");
    printf("3 - Показать матрицу инцидентности\n");
    printf("4 - Вывести размер графа G, изолированные, концевые  
и доминирующие вершины (через матрицу инцидентности)\n");
    printf("5 - Ввести новое количество вершин\n");
    printf("6 - Закрыть программу\n");
    printf("Выберите опцию: ");
}

int main() {
    system("chcp 1251");
    srand(time(NULL));

```

```

int n = 0;
int** adj_matrix = NULL;
int** inc_matrix = NULL;
int edge_count = 0;
double edgeProbability = 0.0;
printf("\n=== Генератор неориентированного графа ===\n\n");
printf("Введите количество вершин: ");
scanf("%d", &n);
if (n <= 0) {
    printf("Ошибка: количество вершин должно быть
положительным числом.\n");
    return 1;
}
edgeProbability = 0.1 + (double)rand() / RAND_MAX * 0.8;
printf("Сгенерированная вероятность ребра: %.2f (%.0f%%)\n",
    edgeProbability, edgeProbability * 100);
adj_matrix = create_adjacency_matrix(n, edgeProbability);
inc_matrix = create_incidence_matrix(adj_matrix, n,
&edge_count);
int choice;
do {
    show_menu();
    scanf("%d", &choice);

    switch (choice) {
        case 1: {
            printf("\n");
            print_matrix(adj_matrix, n);
            break;
        }
        case 2: {
            printf("\n");
            int graph_edges = graph_size(adj_matrix, n);

```

```

        printf("\nРазмер графа G:  $|E(G)| = %d$  ребер\n",
graph_edges);

        print_vertex_degrees(adj_matrix, n);
        find_special_vertices(adj_matrix, n);
        break;
    }

    case 3: {
        printf("\n");
        print_incidence_matrix(inc_matrix, n, edge_count);
        break;
    }

    case 4: {
        printf("\n");
        printf("\nРазмер графа G через матрицу
инцидентности:  $|E(G)| = %d$  ребер\n", edge_count);

        print_vertex_degrees_from_incidence(inc_matrix, n,
edge_count);

        printf("\nСпециальные вершины (через матрицу
инцидентности):\n");

        find_isolated_vertices_from_incidence(inc_matrix, n,
edge_count);

        find_end_vertices_from_incidence(inc_matrix, n,
edge_count);

        find_dominant_vertices_from_incidence(inc_matrix, n,
edge_count);

        break;
    }

    case 5: {
        if (adj_matrix != NULL) free_matrix(adj_matrix, n);
        if (inc_matrix != NULL)
free_incidence_matrix(inc_matrix, n);

        printf("\nВведите новое количество вершин: ");
        scanf("%d", &n);
        if (n <= 0) {

```

```

        printf("Ошибка: количество вершин должно быть
положительным числом.\n");

        return 1;

    }

    edgeProbability = 0.1 + (double)rand() / RAND_MAX *
0.8;

    printf("Сгенерированная вероятность ребра: %.2f
(%.0f%%)\n",

        edgeProbability, edgeProbability * 100);

    adj_matrix = create_adjacency_matrix(n,
edgeProbability);

    inc_matrix = create_incidence_matrix(adj_matrix, n,
&edge_count);

    printf("Новый граф создан успешно!\n");

    break;

}

case 6: {

    printf("\nПрограмма завершена.....\n");

    break;

}

default: {

    printf("Неверный выбор! Пожалуйста, выберите опцию
от 1 до 6.\n");

    break;

}

} while (choice != 6);

if (adj_matrix != NULL) free_matrix(adj_matrix, n);

if (inc_matrix != NULL) free_incidence_matrix(inc_matrix,
n);

return 0;

}

```

Результат работы программы:

Результат работы программы показан на рисунках 1-5

```

=== МЕНЮ ===
1 - Показать матрицу смежности
2 - Вывести размер графа G, изолированные, концевые и доминирующие вершины (через матрицу смежности)
3 - Показать матрицу инцидентности
4 - Вывести размер графа G, изолированные, концевые и доминирующие вершины (через матрицу инцидентности)
5 - Ввести новое количество вершин
6 - Закреть программу
Выберите опцию: 2

Размер графа G:  $|E(G)| = 6$  ребер

Степени вершин (через матрицу смежности):
Вершина 1: степень 0
Вершина 2: степень 3
Вершина 3: степень 2
Вершина 4: степень 2
Вершина 5: степень 0
Вершина 6: степень 1
Вершина 7: степень 0
Вершина 8: степень 1
Вершина 9: степень 2
Вершина 10: степень 1

Специальные вершины (через матрицу смежности):
Изолированные вершины (степень 0): 1 5 7
Концевые вершины (степень 1): 6 8 10
Доминирующие вершины (степень 9): нет

```

Рисунок 1

```

Введите количество вершин: 10
Сгенерированная вероятность ребра: 0.16 (16%)

=== МЕНЮ ===
1 - Показать матрицу смежности
2 - Вывести размер графа G, изолированные, концевые и доминирующие вершины (через матрицу смежности)
3 - Показать матрицу инцидентности
4 - Вывести размер графа G, изолированные, концевые и доминирующие вершины (через матрицу инцидентности)
5 - Ввести новое количество вершин
6 - Закреть программу
Выберите опцию: 1

Матрица смежности (10x10):
  1  2  3  4  5  6  7  8  9 10
1  0  0  0  0  0  0  0  0  0  0
2  0  0  1  1  0  0  0  0  1  0
3  0  1  0  0  0  1  0  0  0  0
4  0  1  0  0  0  0  0  0  0  1
5  0  0  0  0  0  0  0  0  0  0
6  0  0  1  0  0  0  0  0  0  0
7  0  0  0  0  0  0  0  0  0  0
8  0  0  0  0  0  0  0  0  1  0
9  0  1  0  0  0  0  0  0  1  0  0
10 0  0  0  1  0  0  0  0  0  0

```

Рисунок 2

```

=== МЕНЮ ===
1 - Показать матрицу смежности
2 - Вывести размер графа G, изолированные, концевые и доминирующие вершины (через матрицу смежности)
3 - Показать матрицу инцидентности
4 - Вывести размер графа G, изолированные, концевые и доминирующие вершины (через матрицу инцидентности)
5 - Ввести новое количество вершин
6 - Закрыть программу
Выберите опцию: 3

Матрица инцидентности (10 вершин x 6 ребер):
    e1 e2 e3 e4 e5 e6
v1  0  0  0  0  0  0
v2  1  1  1  0  0  0
v3  1  0  0  1  0  0
v4  0  1  0  0  1  0
v5  0  0  0  0  0  0
v6  0  0  0  1  0  0
v7  0  0  0  0  0  0
v8  0  0  0  0  0  1
v9  0  0  1  0  0  1
v10 0  0  0  0  1  0

```

Рисунок 3

```

=== МЕНЮ ===
1 - Показать матрицу смежности
2 - Вывести размер графа G, изолированные, концевые и доминирующие вершины (через матрицу смежности)
3 - Показать матрицу инцидентности
4 - Вывести размер графа G, изолированные, концевые и доминирующие вершины (через матрицу инцидентности)
5 - Ввести новое количество вершин
6 - Закрыть программу
Выберите опцию: 4

Размер графа G через матрицу инцидентности:  $|E(G)| = 6$  ребер

Степени вершин:
Вершина 1: степень 0
Вершина 2: степень 3
Вершина 3: степень 2
Вершина 4: степень 2
Вершина 5: степень 0
Вершина 6: степень 1
Вершина 7: степень 0
Вершина 8: степень 1
Вершина 9: степень 2
Вершина 10: степень 1

Специальные вершины (через матрицу инцидентности):
Изолированные вершины: 1 5 7
Концевые вершины: 6 8 10
Доминирующие вершины: нет

```

Рисунок 4

```

=== МЕНЮ ===
1 - Показать матрицу смежности
2 - Вывести размер графа G, изолированные, концевые и доминирующие вершины (через матрицу смежности)
3 - Показать матрицу инцидентности
4 - Вывести размер графа G, изолированные, концевые и доминирующие вершины (через матрицу инцидентности)
5 - Ввести новое количество вершин
6 - Закрыть программу
Выберите опцию: 5

Введите новое количество вершин: 5
Сгенерированная вероятность ребра: 0.14 (14%)
Новый граф создан успешно!

```

Рисунок 5

Вывод: В ходе выполнения лабораторной работы познакомились с характеристиками графа и научились их определять с помощью программы. Познакомились с алгоритмом задания графа через матрицы.